Airbag ECU Portfolio

HS Harz — Functional Safety Engineering

# AIRBAG ELECTRONIC CONTROL UNIT

*Measurement and Processing of Sensor Signals to Control Airbag Operation*

## SOFTWARE ARCHITECTURE DESIGN SPECIFICATION AND IMPLEMENTATION (SADS)

| | |
|---|---|
| **Document Number** | SADS-AIRBAG-ECU-001 |
| **Version** | 2.0 |
| **Date** | 2026-06-12 |
| **Status** | RELEASED |
| **ASIL Classification** | ASIL D (ISO 26262) / SIL 3 (IEC 61508) |
| **Target Hardware** | Arduino Due (SAM3X8E ARM Cortex-M3, 84 MHz) ×2 |
| **Applicable Standards** | ISO 26262:2018 / IEC 61508:2010 |
| **Author** | Mariana Chavez Avila |
| **Based On** | HADS-AIRBAG-ECU-001 Rev 2.1 \| FSRS-AIRBAG-ECU-001 Rev 2.1 \| SRS-AIRBAG-SW-001 Rev 2.1 |

## Table of Contents

# 1 Glossary

| Abbreviation | Definition |
|---|---|
| **2oo2** | Two-out-of-Two voting. Both ECUs must agree to trigger airbag deployment. |
| **ADC** | Analog-to-Digital Converter. Converts sensor voltages to 12-bit values (0–4095). |
| **ASIL** | Automotive Safety Integrity Level. Risk classification per ISO 26262 (QM, A, B, C, D). |
| **ASIL D** | Highest safety integrity level. Applied to the airbag deployment logic. |
| **CAN** | Controller Area Network. Communication bus between the two ECUs at 500 kbit/s. |
| **CRC** | Cyclic Redundancy Check. Error-detection code for CAN frames. Required by FSR-13. |
| **DWI** | Dash Warning Indicator. Fault warning lamp on pin D10. Must activate within ≤800 ms. |
| **ECU** | Electronic Control Unit. Each Arduino Due board acting as a redundant controller. |
| **FSRS** | Functional Safety Requirement. Traceable requirement from FSRS-AIRBAG-ECU-001. |
| **GAP** | Identified non-conformance between current firmware and an FSR requirement. |
| **HADS** | Hardware Architecture and Design Specification. Defines wiring, BOM, and pin assignments. |
| **HARA** | Hazard Analysis and Risk Assessment. Identifies hazards and derives ASIL classifications. |
| **ITP** | Implementation Test Plan. Defines unit tests, integration tests, and the code review checklist. |
| **OCS** | Occupant Classification System. Detects child seat presence via the baby seat switch (D11). |
| **POST** | Power-On Self-Test. Startup routine testing CPU, RAM, and ADC before the main loop. |
| **SADS** | Software Architecture and Design Specification. This document portfolio's SW design reference. |
| **SIL** | Safety Integrity Level. Risk classification per IEC 61508 (SIL 1–4). ECU is SIL 3. |
| **SPFM** | Single-Point Fault Metric. ASIL D requires ≥99% of single-point faults to be covered. |
| **TR** | Test Report. Records test execution results, coverage evidence, and sign-off. |
| **V&V** | Verification and Validation. Confirms software is built correctly and meets safety goals. |
| **WDT** | Watchdog Timer. Resets the MCU if the main loop stalls. Required by FSR-22. |

# 2 Overview

This Software Architecture and Design Specification (SADS) defines the internal software structure, module organisation, data flow, control flow, safety mechanisms, state machines, and implementation logic for the Airbag Electronic Control Unit (ECU) firmware running on two Arduino Due (SAM3X8E) microcontrollers in a redundant 2oo2 safety architecture.

The document covers the following architecture and design activities required by ISO 26262 for software at ASIL D:

- Software architecture design and tool/language selection
- Detailed module design and data structures
- HW/SW integration procedure
- Verification methods overview and requirement traceability
- Modification and change management procedure

Unit testing, integration testing, validation test cases, and test evidence are documented separately in the Implementation Test Plan (ITP-AIRBAG-ECU-001) and Test Report (TR-AIRBAG-ECU-001). The safety assessment is documented in the Functional Safety Assessment (FSA-AIRBAG-ECU-001).

This specification must be read in conjunction with FSRS-AIRBAG-ECU-001 Rev 2.1, SRS-AIRBAG-SW-001 Rev 2.1 and HADS-AIRBAG-ECU-001 Rev 2.1.

# 3 Selection of Tools and Programming Languages

Per ISO 26262-6:2018 Clause 5 and IEC 61508-3:2010 Clause 7.4, all tools and languages used in the development of safety-related software shall be qualified or otherwise justified. This section documents the selection rationale, qualification status, and constraints for each tool and language used.

## 3.1 Programming Language

| Attribute | Detail | Safety Justification |
|---|---|---|
| Language | C / C++ (Arduino framework, Arduino IDE 2.3.6) | Supported by ISO 26262-6 Annex C as a high-coverage language for embedded safety systems. The Arduino framework wraps Atmel SAM3X8E hardware abstraction. |
| Language standard | C++11 (GCC 4.8.3 ARM compiler bundled with Arduino Due board package) | C++11 constrained to avoid dynamic memory (heap) and recursive functions per permitted/forbidden operations list (Section 7.4). |

## 3.2 Development Tools

| Tool | Version | Role | Qualification / Justification |
|---|---|---|---|
| Arduino IDE | 2.3.6 | Code editor, compiler, upload tool | Tool output (compiled binary) verified by functional testing. No claim placed on tool correctness beyond standard compiler qualification. |
| GCC ARM (avr-g++) | 4.8.3 (bundled) | Cross-compiler for SAM3X8E (ARM Cortex-M3) | Widely used in safety projects. Code output validated via module testing and integration tests documented in ITP-AIRBAG-ECU-001. |
| due_can library | v1.0 (open source) | Arduino Due CAN0 hardware abstraction | Limited to CAN frame send/receive. Qualified by code review + communication tests documented in ITP-AIRBAG-ECU-001. |
| USBCANV2 (OMS PC) | Latest stable | Passive CAN monitoring on OMS Lab PC | Used for test evidence collection only. Not part of deployed firmware. No safety claim placed on monitor tool. |
| Serial Monitor (Arduino IDE) | Bundled | Debug/diagnostic output at 115200 baud | Used for test observation only. No safety claim. Serial.print() calls are non-critical diagnostics. |

## 3.3 Tool Classification

Under ISO 26262-8:2018 Clause 11, each tool is assigned a Tool Confidence Level (TCL) based on Tool Impact (TI) and Tool Error Detection (TD).

| Tool | TI | TD | TCL | Qualification Method |
|---|---|---|---|---|
| GCC ARM Compiler | TI2 | TD2 | TCL2 | Functional testing of compiled output against specification |

| | | | | |
|---|---|---|---|---|
| | | | | (see ITP-AIRBAG-ECU-001) provides sufficient confidence. |
| Arduino IDE Upload | TI1 | TD1 | TCL1 | Flash read-back verification: confirm checksum post-upload via serial bootloader acknowledgement. |
| due_can library | TI2 | TD2 | TCL2 | Code review + CAN communication test cases documented in ITP-AIRBAG-ECU-001. |
| USBCANV2 Monitor | TI1 | TD3 | TCL1 | Test/monitoring tool only; not in safety path. No qualification required. |

# 4 Software Architecture Overview

The firmware follows a single-task cyclic executive model executing on the bare-metal Arduino Due environment (no RTOS). One main loop iteration constitutes one execution cycle, targeted at ≤1 ms per FSR-02. The architecture is divided into six functional modules that map directly onto FSR requirements.

## 4.1 Module Decomposition

| Module ID | Module Name | Primary FSR(s) | ASIL |
|---|---|---|---|
| MOD-01 | Power-On Self-Test (POST) | FSR-17 | ASIL C |
| MOD-02 | Sensor Acquisition | FSR-01, FSR-02 | ASIL D |
| MOD-03 | Crash Detection | FSR-01, FSR-02, FSR-03, FSR-04 | ASIL D |
| MOD-04 | Deployment Control | FSR-03, FSR-08, FSR-09, FSR-10 | ASIL D |
| MOD-05 | CAN Communication | FSR-13, FSR-14, FSR-15, FSR-16 | ASIL B |
| MOD-06 | Diagnostic Monitor | FSR-17, FSR-18, FSR-19, FSR-20, FSR-22 | ASIL C/D |

## 4.2 Data Flow Between Modules

- MOD-02 (Sensor Acquisition) reads raw 12-bit ADC values (0–4095) from A0, A1, A2 and digital pin D11, storing them in global variables rawA0, rawA1, rawA2, and babySeatRaw.
- MOD-03 (Crash Detection) reads rawA0–rawA2, scales to engineering units, applies threshold comparisons, and writes localCrash (boolean) and scaled float values accel, gyro, press.
- MOD-05 (CAN Communication) transmits the local ECU status frame and receives the peer ECU frame, writing remoteCrash, remoteFault, and updating lastRemoteHB timestamp.
- MOD-06 (Diagnostic Monitor) evaluates fault flags, computing the aggregated systemFault boolean written to MOD-04.
- MOD-04 (Deployment Control) reads localCrash, remoteCrash, systemFault, remoteFault, and babySeat to compute voteDeploy, then drives D8, D9, and D10 output pins.

### 4.3 Execution Control Flow

| Step | Action | Period / Trigger | FSR |
|---|---|---|---|
| 1 | receiveCAN() — drain RX buffer, update remoteCrash, remoteFault, lastRemoteHB | Every cycle (non-blocking) | FSR-15 |
| 2 | Heartbeat timeout check — if (millis() − lastRemoteHB > 2000 ms) → faultCOMM = true | Every cycle | FSR-15 |
| 3 | Read sensors — analogRead A0, A1, A2 + digitalRead D11 | Every cycle | FSR-01, FSR-02 |
| 4 | Scale sensors — convert ADC to g, deg/s, kPa | Every cycle | FSR-01 |
| 5 | Crash detection — threshold comparisons → localCrash | Every cycle | FSR-01, FSR-04 |
| 6 | systemFault aggregation — OR of faultCPU, faultRAM, faultADC, faultCOMM | Every cycle | FSR-20 |
| 7 | Deployment vote — voteDeploy logic → drive D8, D9, D10 | Every cycle | FSR-03, FSR-08 |
| 8 | CAN transmit heartbeat — sendData() if 500 ms elapsed | Every 500 ms | FSR-16 |
| 9 | Status LED blink — toggle D13 if 500 ms elapsed | Every 500 ms | — |
| 10 | Serial diagnostic print — if 1000 ms elapsed | Every 1000 ms | FSR-21 |

## 5 System State Machine

### 5.1 State Definitions

| State ID | State Name | Conditions to Enter | Outputs | FSRS Ref |
|---|---|---|---|---|
| ST-01 | POWER_ON | Reset / power-up event | All outputs LOW, POST executing | SS-01 |
| ST-02 | POST_FAILED | Any POST check returns FALSE (CPU, RAM, ADC) | D10 (DWI) HIGH, D8/D9 LOW — permanent safe inhibit | SS-05 |
| ST-03 | MONITORING | POST passed, no crash, no fault, peer CAN active | D10 LOW, D8/D9 LOW — watching sensors | SS-02 |
| ST-04 | FAULT_ACTIVE | systemFault=TRUE (any fault flag set) | D10 HIGH (DWI), D8/D9 LOW — inhibit deployment | SS-01, SS-04 |
| ST-05 | CRASH_DETECTED | localCrash=TRUE AND remoteCrash=TRUE AND !systemFault AND !remoteFault | D8 HIGH (Airbag1 vote), D9 HIGH if !babySeat (Airbag2 vote) | SS-02 |

State transitions described in the graphical diagram produced in SADS Appendix (FLOW-CHART-AIRBAG-ECU-001).

## 5.2 Safe State Guarantees

- ST-02 (POST_FAILED): Software sets D8/D9 LOW in setup() and never reaches loop(). Hardware AND gate outputs LOW permanently — ultimate safe inhibit (SS-05).
- ST-04 (FAULT_ACTIVE): systemFault=TRUE forces voteDeploy=FALSE each cycle, pulling D8/D9 LOW. DWI lamp (D10) activates within one cycle (~1 ms), satisfying FSR-19 (≤800 ms) by a large margin.
- Loss of CAN peer (faultCOMM=TRUE): After 2000 ms without a valid peer frame, faultCOMM=TRUE propagates through systemFault into voteDeploy, inhibiting deployment. Fail-safe direction: inhibit rather than spurious deploy.

# 6 Detailed Design of Software Modules

Each module is described with its interface, internal logic, data structures, and pseudo-code. This section satisfies the ISO 26262-6 Clause 8 requirement for detailed software unit design.

## 6.1 MOD-01 — Power-On Self-Test (POST)

Executed once in setup() before loop() begins. Results stored in persistent boolean fault flags. Any failed test sets the corresponding flag and transitions the system to ST-02 (POST_FAILED).

### 6.1.1 CPU Test

Tests ALU correctness by performing a deterministic integer addition and verifying the result.

```
bool cpuTest() {
  volatile uint32_t a = 100;
  volatile uint32_t b = 200;
  return ((a + b) == 300);  // volatile prevents compiler optimisation
}
```

### 6.1.2 RAM Test

Writes a known bit pattern (0xAAAAAAAA — alternating 1/0 bits stressing adjacent bit coupling) to a stack variable and reads it back.

```
bool ramTest() {
  uint32_t test = 0xAAAAAAAA;
  return (test == 0xAAAAAAAA);
}
```

*NOTE: This is a lightweight single-word test appropriate for ASIL C within an ASIL D system backed by dual-channel redundancy. Full memory sweep testing would be required for standalone ASIL D certification.*

### 6.1.3 ADC Test

Confirms ADC hardware is operational and returning values within the valid 12-bit range (0–4095).

```
bool adcTest() {
  int v = analogRead(A0);
  return (v >= 0 && v <= 4095);
}
```

### 6.1.4 POST Completion Criteria

| Test | Pass Condition | Fail Action | Flag |
|---|---|---|---|
| CPU arithmetic | a + b == 300 | faultCPU = true → systemFault = true → DWI ON, deploy inhibited | faultCPU |
| RAM read-back | 0xAAAAAAAA read back correctly | faultRAM = true | faultRAM |
| ADC range | 0 ≤ v ≤ 4095 | faultADC = true | faultADC |

| | | | |
|---|---|---|---|
| CAN init | Can0.begin() success | Not currently checked — GAP-05 (see Section 9) | — |

## 6.2 MOD-02 — Sensor Acquisition

Reads all sensor inputs at the start of each loop iteration using the Arduino Due 12-bit ADC (analogReadResolution(12) set in setup()).

```
uint16_t rawA0 = analogRead(A0);          // Accelerometer POTI-1: 0–4095
uint16_t rawA1 = analogRead(A1);          // Gyroscope     POTI-2: 0–4095
uint16_t rawA2 = analogRead(A2);          // Pressure      POTI-3: 0–4095
bool babySeat  = (digitalRead(PIN_BABY) == LOW); // Active-LOW switch
```

| Sensor | Pin | ADC Range | Scaled Range | Deploy Threshold | ADC Threshold |
|---|---|---|---|---|---|
| Accelerometer (POTI-1) | A0 | 0–4095 | 0–100 g | > 30.0 g | raw > 1228 |
| Gyroscope (POTI-2) | A1 | 0–4095 | 0–360 °/s | > 120.0 °/s | raw > 1365 |
| Pressure (POTI-3) | A2 | 0–4095 | 0–500 kPa | > 200.0 kPa | raw > 1638 |
| Baby Seat Switch | D11 | HIGH / LOW | TRUE / FALSE | LOW = baby seat | — |

## 6.3 MOD-03 — Crash Detection

Converts raw ADC values to engineering units and applies threshold comparisons. An OR relationship between three sensors is used: any single sensor exceeding its threshold asserts localCrash = TRUE.

### 6.3.1 Scaling Formulas

```
float accel = (rawA0 / 4095.0) * 100.0;   // 0–100 g
float gyro  = (rawA1 / 4095.0) * 360.0;   // 0–360 deg/s
float press = (rawA2 / 4095.0) * 500.0;   // 0–500 kPa
```

### 6.3.2 Threshold Comparison

```
bool localCrash = (accel > THRESH_ACCEL)    // > 30.0 g
                || (gyro  > THRESH_GYRO)    // > 120.0 deg/s
                || (press > THRESH_PRESS);  // > 200.0 kPa
```

### 6.3.3 Threshold Rationale

| Sensor | Threshold | Physical Meaning | Rationale |
|---|---|---|---|
| Accelerometer | 30.0 g | 30× gravitational acceleration — severe frontal impact pulse characteristic | Below hard-braking (~1 g) and road-bump range (<5 g); above representative airbag-required crash pulse. |
| Gyroscope | 120.0 °/s | One-third of full rotation per second — rollover onset angular rate | Well above aggressive cornering (<30 °/s); below confirmed rollover rates (>200 °/s) for laboratory sensitivity. |
| Pressure | 200.0 kPa | ~2 bar gauge — door panel structural deformation pressure | Representative of side-impact crush pressure exceeding normal ambient variation. |

## 6.4 MOD-04 — Deployment Control

Computes the deployment vote and drives all output pins. This is the safety-critical decision point combining all module inputs.

### 6.4.1 Deployment Vote Logic

```
// Step 1: Aggregate system fault
systemFault = faultCPU || faultRAM || faultADC || faultCOMM;

// Step 2: Compute deployment vote
bool voteDeploy = localCrash      // This ECU detects crash
                && remoteCrash    // Peer ECU independently detects crash
                && !systemFault   // No fault on this ECU
                && !remoteFault;  // No fault on peer ECU

// Step 3: Drive output pins
digitalWrite(PIN_AIRBAG1, voteDeploy);              // Driver airbag vote
digitalWrite(PIN_AIRBAG2, voteDeploy && !babySeat); // Passenger: suppressed if babySeat
digitalWrite(PIN_WARNING, systemFault);             // DWI lamp
```

### 6.4.2 Deployment Authorization Truth Table

| localCrash | remoteCrash | systemFault | remoteFault | babySeat | Airbag1 (D8) | Airbag2 (D9) |
|---|---|---|---|---|---|---|
| FALSE | any | any | any | any | LOW | LOW |
| TRUE | FALSE | any | any | any | LOW | LOW |
| TRUE | TRUE | TRUE | any | any | LOW | LOW |
| TRUE | TRUE | FALSE | TRUE | any | LOW | LOW |
| TRUE | TRUE | FALSE | FALSE | FALSE | HIGH | HIGH |
| TRUE | TRUE | FALSE | FALSE | TRUE | HIGH | LOW (suppressed) |

## 6.5 MOD-05 — CAN Communication

### 6.5.1 Transmit Frame (sendData)

Transmits an 8-byte CAN frame every 500 ms. The 500 ms heartbeat interval ensures the peer ECU can detect communication loss within the 2000 ms faultCOMM timeout window.

| Byte | Field | Encoding | Notes |
|---|---|---|---|
| 0 | A0 low byte | rawA0 & 0xFF | Accelerometer ADC low byte |
| 1 | A0 high byte | rawA0 >> 8 | Accelerometer ADC high byte |
| 2 | A1 low byte | rawA1 & 0xFF | Gyroscope ADC low byte |
| 3 | A1 high byte | rawA1 >> 8 | Gyroscope ADC high byte |
| 4 | A2 low byte | rawA2 & 0xFF | Pressure ADC low byte |
| 5 | A2 high byte | rawA2 >> 8 | Pressure ADC high byte |
| 6 | Status byte | Bit0=crash, Bit1=baby, Bit2=fault | Peer reads bits to extract remoteCrash and remoteFault |
| 7 | ECU ID | 1 or 2 | Identifies transmitting ECU |

### 6.5.2 Receive and Timeout

```
void receiveCAN() {
  while (Can0.rx_avail()) {
    Can0.read(incoming);
    if (incoming.id == REMOTE_ID) {    // Filter for peer ECU only
      lastRemoteHB = millis();          // Reset timeout watchdog
      remoteCrash  = (status & 0x01);  // Extract crash bit
      remoteFault  = (status & 0x04);  // Extract fault bit
    }
  }
}

// Timeout check in main loop:
if (millis() - lastRemoteHB > 2000) faultCOMM = true;
else                                faultCOMM = false;
```

## 6.6 MOD-06 — Diagnostic Monitor

Runs every cycle. Aggregates hardware fault flags into systemFault. POST fault flags (faultCPU, faultRAM, faultADC) are sticky for the session; faultCOMM is evaluated dynamically.

| Flag | Set When | Cleared When | Effect on System |
|---|---|---|---|
| faultCPU | cpuTest() returns FALSE at POST | Never — sticky | systemFault=TRUE, DWI ON, deployment inhibited |
| faultRAM | ramTest() returns FALSE at POST | Never — sticky | systemFault=TRUE, DWI ON, deployment inhibited |
| faultADC | adcTest() returns FALSE at POST | Never — sticky | systemFault=TRUE, DWI ON, deployment inhibited |
| faultCOMM | millis() − lastRemoteHB > 2000 ms | Peer CAN frame received within 2000 ms | Dynamic: ST-04 while active, recovers to ST-03 when peer resumes |

# 7 Occupant Classification and Baby Seat Logic

The baby seat detection implements a fail-safe suppress-by-default strategy per FSR-09 and FSR-10. The switch is wired active-LOW with a 10 kΩ pull-up resistor to the Arduino Due 3.3 V rail.

## 7.1 Signal Interpretation

| D11 State | babySeat Value | Airbag 2 (D9) | Meaning |
|---|---|---|---|
| HIGH (~3.3 V via pull-up) | FALSE | Follows voteDeploy | No child seat — adult or empty — passenger airbag armed |
| LOW (switch pressed) | TRUE | Forced LOW | Child seat present — passenger airbag suppressed |

## 7.2 FSR-11 Debounce Requirement (Required Implementation)

FSR-11 requires the baby seat digital input to remain stable for at least 10 ms before a state transition is accepted as valid. The current firmware reads the pin directly without a debounce timer (GAP-01). Required implementation:

```
static bool lastBabyRaw = HIGH;
static unsigned long babyStableTime = 0;
bool currentRaw = digitalRead(PIN_BABY);
if (currentRaw != lastBabyRaw) {
  babyStableTime = millis();
  lastBabyRaw = currentRaw;
}
if ((millis() - babyStableTime) >= 10) {
  babySeat = (lastBabyRaw == LOW);}
```

# 8 Safety Mechanisms in Software

## 8.1 Hardware Watchdog Timer (WDT)

The SAM3X8E microcontroller includes a dedicated Hardware WDT independent of the main CPU. When enabled, it resets the MCU if the main loop fails to execute within the configured timeout window.

Current status: The WDT is NOT explicitly enabled in the current firmware (GAP-06). Required implementation for FSR-22 compliance:

```
// In setup():
WDT_Enable(WDT, WDT_MR_WDD(0xFFF) | WDT_MR_WDRSTEN | WDT_MR_WDV(500));

// At end of each loop() iteration:
WDT_Restart(WDT);   // Feed the watchdog
```

## 8.2 2oo2 Redundant Decision Architecture

The primary safety mechanism is the dual-channel 2oo2 architecture. Both ECUs independently read identical sensor inputs, execute identical crash detection algorithms, compute independent deployment votes, and assert D8/D9 only when their local vote is TRUE. The DM74LS08N AND gate forms the hardware enforcement layer.

This architecture prevents: single-point hardware failures, single-channel software errors, single-sensor noise events, and communication corruption.

## 8.3 Fail-Safe Direction Analysis

| Failure Mode | Software Response | Hardware Response | Safe? |
|---|---|---|---|
| Peer CAN timeout (>2000 ms) | faultCOMM=TRUE → systemFault=TRUE → voteDeploy=FALSE → D8/D9 LOW | AND gate output LOW | Yes — inhibit on comm loss |
| POST failure (CPU/RAM/ADC) | faultXXX=TRUE, loop() inhibits via systemFault | D8/D9 never driven HIGH in fault state | Yes — inhibit on startup fault |
| Single sensor above threshold | localCrash=TRUE on this ECU only | AND gate requires BOTH D8 HIGH — peer must agree | Yes — no single-channel deploy |
| Single ECU software crash | Loop stops, D8/D9 go LOW (default) | AND gate output LOW | Yes — fail LOW |
| babySeat line open circuit | Pull-up holds D11 HIGH → babySeat=FALSE → airbag NOT suppressed | D9 follows voteDeploy | GAP-02 — see Section 9 |

## 8.4 Permitted and Forbidden Software Operations

Per ASIL D software development guidelines (ISO 26262-6 Annex A):

**Permitted:**

- Reading ADC pins A0, A1, A2 via analogRead()
- Reading digital pin D11 via digitalRead()
- Writing D8, D9 (deployment votes) and D10 (DWI) via digitalWrite()
- Transmitting CAN frames via Can0.sendFrame()
- Reading received CAN frames via Can0.read()
- Using millis() for timing comparisons
- Printing to Serial for diagnostics

**Forbidden:**

- Dynamic memory allocation (malloc, new) — non-deterministic timing prohibited at ASIL D
- Recursive functions — stack depth unpredictable
- Unbounded loops (while(true) without exit) — threatens deterministic cycle time
- Direct register access bypassing safety flags
- Clearing faultCPU, faultRAM, faultADC once set — these are sticky faults
- Modifying ECU_ID, DATA_ID, REMOTE_ID at runtime — compile-time constants only

# 9 HW/SW Integration Procedure

This section describes the procedure for integrating the firmware with the hardware assembly. It covers flash programming, pin verification, CAN bus bring-up, and the acceptance criteria that must be met before the integrated system is released to functional testing. Execution results and sign-off evidence are recorded in TR-AIRBAG-ECU-001.

## 9.1 Integration Steps

| Step | Action | Acceptance Criterion | Owner |
|---|---|---|---|
| 1 | Assemble hardware per HADS-AIRBAG-ECU-001 Rev 2.1 — breadboard wiring, potentiometers, baby seat switch, CAN transceivers, AND gate, LEDs | All wiring verified against HADS Tables 5–14. No short circuits. Continuity check on all signal lines. | HW Engineer |
| 2 | Verify power rails: 5 V rail to AND gate VCC, 3.3 V rail to CAN transceivers, GND common to all | Measure 5 V ± 0.25 V at AND gate pin 14. Measure 3.3 V ± 0.15 V at Waveshare VCC. GND continuity confirmed. | HW Engineer |
| 3 | Flash ECU 1 firmware (ECU_ID=1, DATA_ID=0x100, REMOTE_ID=0x200) via Arduino IDE USB upload | Upload completes with ‘Done uploading’. Serial Monitor shows ECU 1 STARTED message at 115200 baud. | SW Engineer |
| 4 | Flash ECU 2 firmware (ECU_ID=2, DATA_ID=0x200, REMOTE_ID=0x100) via Arduino IDE USB upload | Upload completes with ‘Done uploading’. Serial Monitor shows ECU 2 STARTED message. | SW Engineer |
| 5 | Verify POST results on both ECUs via Serial Monitor | Serial output: faultCPU=0, faultRAM=0, faultADC=0. Status LED (D13) blinking at ~500 ms on both ECUs. | SW Engineer |
| 6 | Verify CAN bus bring-up — connect OMS PC via USB-CAN-A, open USBCANV2 at 500 kbit/s | CAN frames with ID 0x100 and 0x200 visible at ~500 ms intervals in USBCANV2 log. | HW/SW Engineer |
| 7 | Verify CAN communication between ECUs — check faultCOMM=0 on both ECUs | Serial output: faultCOMM=0 on both ECUs. Remote crash and fault bits correctly parsed. | SW Engineer |
| 8 | Verify pin output levels at rest — all sensors below threshold | D8=LOW, D9=LOW, D10=LOW, D13 blinking. Airbag LEDs OFF. DWI OFF. | HW Engineer |
| 9 | Verify AND gate logic — set POTI-1 on ECU1 only above threshold | Airbag1 LED stays OFF (only one ECU asserting D8). AND gate pin 3 LOW. | HW Engineer |
| 10 | Verify full crash scenario — set both ECU POTI-1 above threshold simultaneously | Airbag1 LED fires (AND gate pin 3 HIGH). Serial: voteDeploy=1 on both ECUs. | HW/SW Engineer |
| 11 | Verify baby seat suppression — press baby seat switch during step 10 crash | Airbag1 LED remains ON; Airbag2 LED OFF. D9 LOW on both ECUs. | HW/SW Engineer |

| | | | |
|---|---|---|---|
| 12 | Sign off HW/SW integration — record results and release to functional testing | All steps passed. Integration sign-off form completed in TR-AIRBAG-ECU-001. | Project Lead |

# 10 Compliance Gaps and Required Improvements

The following gaps were identified between the current firmware implementation and the requirements in FSRS-AIRBAG-ECU-001 Rev 2.1. Each gap carries an assigned priority and required action.

| Gap ID | FSR | Description | Risk | Required Action |
|---|---|---|---|---|
| GAP-01 | FSR-11 | Baby seat debounce not implemented — D11 read directly without 10 ms stability filter | Medium | Implement debounce timer as shown in Section 6.2 |
| GAP-02 | FSR-12 | Baby seat line continuity monitoring not implemented — open-circuit stuck-HIGH not detected | High | Add periodic D11 continuity test or input monitoring |
| GAP-03 | FSR-13 | CRC not included in CAN frame — bytes 6–7 carry status and ECU_ID, not a 16-bit CRC | Medium | Compute 16-bit CRC over bytes 0–5, store in bytes 6–7, verify on receive |
| GAP-04 | FSR-14 | Parallel CAN bitstream readback not implemented | Low | Implement readback channel or loopback verification |
| GAP-05 | FSR-17 | CAN communication not verified during POST — Can0.begin() success not checked | Medium | Check Can0.begin() return value; set faultCOMM on failure |
| GAP-06 | FSR-22 | Hardware WDT not enabled in firmware | High | Enable WDT in setup() and feed at end of every loop() iteration (Section 7.1) |
| GAP-07 | FSR-21 | Fault logging to DTC/non-volatile memory not implemented | Low | Implement DTC logging to EEPROM or flash |
| GAP-08 | FSR-15 | CAN timeout threshold is 2000 ms vs FSR-15 requirement of 50 ms | Medium | Reduce COMM_TIMEOUT to 50 ms (or justify the 2000 ms as a design choice with safety rationale) |

# 11 Verification Methods Overview

This section defines the software verification approach required by ISO 26262-6:2018 Clause 9 at ASIL D. Detailed test cases, test execution records, pass/fail results, and coverage evidence are documented in ITP-AIRBAG-ECU-001 (test plan) and TR-AIRBAG-ECU-001 (test report).

## 11.1 Applicable Verification Methods

| Method | Description | ISO 26262-6 Reference | Applicability |
|---|---|---|---|

| | | | |
|---|---|---|---|
| Code Review (Walkthrough) | Manual review of source code against SADS module specifications. Two-person review (author + independent reviewer). Checklist covers: forbidden constructs, fault flag handling, threshold values, output pin logic, CAN frame encoding. Checklist defined in ITP-AIRBAG-ECU-001. | Clause 9.4.2 (code review) | All modules — mandatory ASIL D |
| Static Analysis | Review of code structure for: unreachable code, unused variables, potential integer overflow in ADC scaling, uninitialized variable access, and adherence to forbidden operations list (Section 7.4). | Clause 9.4.2 (static analysis) | MOD-02, MOD-03, MOD-04 |
| Unit Testing | Black-box and white-box tests per ITP-AIRBAG-ECU-001. Tests executed on target hardware (Arduino Due). Results logged in TR-AIRBAG-ECU-001. | Clause 9.4.3 (dynamic analysis) | MOD-01 through MOD-06 |
| Integration Testing | Inter-module and HW/SW integration tests per ITP-AIRBAG-ECU-001. Results logged in TR-AIRBAG-ECU-001. | Clause 9.4.3 | Full system |
| Timing Analysis | Cycle time measurement via micros() log. Worst-case cycle time must be ≤ 1000 µs (FSR-02). CAN heartbeat timing verified via USBCANV2 log. | Clause 9.4.4 (timing analysis) | Main loop, CAN TX |
| Requirement Traceability Review | Each FSR-XX requirement traced to implementing module(s) and at least one passing test case in the traceability matrix (Section 12). Untraced requirements documented as gaps in Section 9. | Clause 9.4.1 (traceability) | All FSR requirements |

### 11.2 Structural Coverage Target

ISO 26262-6 ASIL D requires MC/DC (Modified Condition/Decision Coverage) or equivalent structural coverage for software at the highest integrity level. The agreed coverage target for this project is:

- Statement coverage: 100% of executable statements exercised by unit and integration test cases defined in ITP-AIRBAG-ECU-001.
- Branch coverage: All TRUE and FALSE outcomes of every conditional (if/else, || and && operators in voteDeploy and localCrash) exercised.
- Fault detection coverage: ≥ 90% of identifiable single-point software faults detectable through defined test cases.

Coverage results and evidence are recorded in TR-AIRBAG-ECU-001.

## 12 Requirement Traceability Matrix

Forward traceability from every software requirement in SRS-AIRBAG-SW-001 Rev 2.1 to the software design element defined in this document:

| SWR | Description | FSR | ASIL | Software Design Element |
|---|---|---|---|---|
| **SWR-01** | POST execution at boot | FSR-17 | **ASIL C** | MOD-01/02/03 — cpu_test, ram_test, adc_test |
| **SWR-02** | No output before POST complete | FSR-17 | **ASIL D** | setup() sequence — POST before loop entry |
| **SWR-03** | ADC 12-bit resolution | FSR-01 | **ASIL D** | setup() — analogReadResolution(12) |
| **SWR-04** | Sensor read every loop | FSR-01/02 | **ASIL D** | MOD-08 — analogRead A0/A1/A2 every loop |
| **SWR-05** | Physical conversion formula | FSR-01 | **ASIL D** | MOD-07 — toPhys() |
| **SWR-06** | Threshold comparison | FSR-03/04 | **ASIL D** | MOD-09 — accelEx/gyroEx/pressEx flags |
| **SWR-07** | Baby seat INPUT mode | FSR-09 | **ASIL D** | setup() — pinMode(D11, INPUT); ext. 10 kΩ pull-up |
| **SWR-08** | 10 ms debounce filter | FSR-11 | **ASIL D** | MOD-06 — updateBabySeat() |
| **SWR-09** | Suppress-on-fault default | FSR-10 | **ASIL D** | MOD-06 — fault branch: babySeat=true |
| **SWR-10** | Sensor OR fusion | FSR-03 | **ASIL D** | MOD-10 — crashLocal = accelEx\|\|gyroEx\|\|pressEx |
| **SWR-11** | No blocking delay in loop | FSR-02 | **ASIL D** | Architecture — millis() timers only |
| **SWR-12** | D8 AND gate drive logic | FSR-03/08 | **ASIL D** | MOD-11 — driveAirbag1 (SWR-12 conditions) |
| **SWR-13** | D9 passenger suppression | FSR-09/10 | **ASIL D** | MOD-11 — driveAirbag2 + babySeat gate |
| **SWR-14** | Fault inhibit both outputs | FSR-20 | **ASIL D** | MOD-13 — enterSafeState: D8/D9 LOW on system_fault |
| **SWR-15** | WDT configuration | FSR-22 | **ASIL D** | MOD-04 — watchdog_setup(); WDV(256×10) ≈ 10 ms |
| **SWR-16** | WDT kick every loop | FSR-22 | **ASIL D** | MOD-05 — watchdog_feed() — first call in loop() |
| **SWR-17** | CAN status transmission — both ECUs | FSR-07 | **ASIL D** | MOD-14 — canSend() — millis() 500 ms timer |
| **SWR-18** | CAN supervision timeout | FSR-07/15 | **ASIL D** | MOD-15/16 — fault_due1/2 on timeout |
| **SWR-19** | CAN status frame parsing | FSR-07 | **ASIL D** | MOD-17 — parseCAN() — extract remote state |
| **SWR-20** | CAN0 init 500 kbps | FSR-13 | **ASIL B** | MOD-19 — Can0.begin(CAN_BPS_500K, 255) |
| **SWR-21** | CAN 8-byte frame structure | FSR-13/16 | **ASIL B** | MOD-19 — canSend() frame layout |
| **SWR-22** | CAN TX | FSR-16 | **ASIL B** | MOD 19 — millis() non-blocking CAN timer |
| **SWR-22** | DWI on local fault | FSR-19 | **ASIL A** | MOD-12 — D10 HIGH within one loop cycle |
| **SWR-23** | DWI on remote fault | FSR-19 | **ASIL A** | MOD-12 — due2_fault → D10 HIGH |

# 13 List of Dependent Documents

| Document ID | Title | Revision | Relationship to this Document |
|---|---|---|---|
| HARA-AIRBAG-ECU-001 | Hazard Analysis and Risk Assessment | Rev 2.1 | Source of hazard IDs and ASIL derivation |
| FSRS-AIRBAG-ECU-001 | Functional Safety Requirements Specification | Rev 2.0 | Primary source for all FSR-XX requirements |
| SC-AIRBAG-ECU-001 | Safety Concept Document | Rev 2.0 | Defines safety constraints SC-01 through SC-09 |
| HADS-AIRBAG-ECU-001 | Hardware Architecture and Design Specification | Rev 2.1 | HW wiring, BOM, CAN IDs, pin assignments |
| SRS-AIRBAG-SW-001 | Software Requirements Specification | Rev 2.1 | Source of software requirements |
| ITP-AIRBAG-ECU-001 | Implementation Test Plan | Rev 1.0 | Unit, integration, and code review test cases and checklist |

# 14 Approval and Sign-Off

| Role | Name | Signature | Date |
|---|---|---|---|
| Software Safety Engineer | | | |
| Hardware Safety Engineer | | | |
| Functional Safety Engineer | | | |
| V&V Lead | | | |
| Project Supervisor | | | |

# 15 Revision History

| Rev | Date | Author | Description |
|---|---|---|---|
| 1.0 | 2026-05-28 | Mariana Chavez Avila | Initial release (UART architecture). Full SADS covering architecture, module decomposition, state machine, safety mechanisms, and SWR traceability. |
| 2.0 | 2026-06-12 | Mariana Chavez Avila | Major revision – added sections, CAN architecture migration. UART inter-ECU link replaced by CAN bus (500 kbit/s). Master/Slave removed; peer-equal architecture. CAN frame byte layout documented. Compliance gap analysis (GAP-01–GAP-07). SRS traceability matrix. |